PYTHON PROGRAMMING LANGUAGE

Anitya Gupta

Inheritance

- Inheritance is a fundamental concept in object-oriented programming (OOP) that allows a class (called a child or derived class) to inherit attributes and methods from another class (called a parent or base class).

- This promotes code reuse, modularity, and a hierarchical class structure.

Example

Inheritance allows us to define a class that inherits all the methods and properties from another class.

```
# Parent class
class Animal:
    def __init__(self, name):
        self.name = name  # Initialize the name attribute

    def speak(self):
        pass  # Placeholder method to be overridden by child classes

# Child class inheriting from Animal
class Dog(Animal):
    def speak(self):
        return f"{self.name} barks!"  # Override the speak method

# Creating an instance of Dog
dog = Dog("Buddy")
print(dog.speak())
```

Output

```
Buddy barks!
```

Explanation:

- Animal is the parent class with an __init__ method and a speak method.
- Dog is the child class that inherits from Animal.
- The speak method is overridden in the Dog class to provide specific behavior.

Syntax of Inheritance

```
class ParentClass:

    # Parent class code here

    pass

class ChildClass(ParentClass):

    # Child class code here

    pass
```

Parent vs Child Class

Parent Class:

1. This is the base class from which other classes inherit.
2. It contains attributes and methods that the child class can reuse.

Child Class:

2. This is the derived class that inherits from the parent class.
3. The syntax for inheritance is class ChildClass(ParentClass).
4. The child class automatically gets all attributes and methods of the parent class unless overridden.

Creating Parent Class

```python
# A Python program to demonstrate inheritance
class Person(object):

  # Constructor
  def __init__(self, name, id):
    self.name = name
    self.id = id

  # To check if this person is an employee
  def Display(self):
    print(self.name, self.id)


# Driver code
emp = Person("Satyam", 102) # An Object of Person
emp.Display()
```

Output

```
Satyam 102
```

Explanation:

- The Person class has two attributes: name and id. These are set when an object of the class is created.
- The display method prints the name and id of the person.

Creating Child Class

In this case, Emp is the child class that inherits from the Person class:

```
class Emp(Person):

  def Print(self):
    print("Emp class called")

Emp_details = Emp("Mayank", 103)

# calling parent class function
Emp_details.Display()

# Calling child class function
Emp_details.Print()
```

Explanation:

- **Emp class** inherits the name and id attributes and the display method from the Person class.
- **__init__ method** in Emp calls super().__init__(name, id) to invoke the constructor of the Person class and initialize the inherited attributes.
- **Emp** introduces an additional attribute, role, and also overrides the display method to print the role in addition to the name and id.

__init__() Function

__init__() function is a constructor method in Python. It initializes the object's state when the object is created. If the child class does not define its own __init__() method, it will automatically inherit the one from the parent class.

In the example above, the __init__() method in the Employee class ensures that both inherited and new attributes are properly initialized.

```
# Parent Class: Person
class Person:
    def __init__(self, name, idnumber):
        self.name = name
        self.idnumber = idnumber

# Child Class: Employee
class Employee(Person):
    def __init__(self, name, idnumber, salary, post):
        super().__init__(name, idnumber)  # Calls Person's __init__()
        self.salary = salary
        self.post = post
```

Explanation:

- **__init__() method** in Person initializes name and idnumber.
- __init__() method in Employee calls **super().__init__(name, idnumber)** to initialize the name and idnumber inherited from the Person class and adds salary and post.

Super() function

super() function is used to call the parent class's methods. In particular, it is commonly used in the child class's __init__() method to initialize inherited attributes. This way, the child class can leverage the functionality of the parent class.

Example:

```
# Parent Class: Person
class Person:
    def __init__(self, name, idnumber):
        self.name = name
        self.idnumber = idnumber

    def display(self):
        print(self.name)
        print(self.idnumber)

# Child Class: Employee
class Employee(Person):
    def __init__(self, name, idnumber, salary, post):
        super().__init__(name, idnumber)  # Using super() to call Person's __init__()
        self.salary = salary
        self.post = post
```

Explanation:

- The super() function is used inside the __init__() method of Employee to call the constructor of Person and initialize the inherited attributes (name and idnumber).
- This ensures that the parent class functionality is reused without needing to rewrite the code in the child class.

Next class

1. **Single Inheritance**: A child class inherits from one parent class.
2. **Multiple Inheritance**: A child class inherits from more than one parent class.
3. **Multilevel Inheritance**: A class is derived from a class which is also derived from another class.
4. **Hierarchical Inheritance**: Multiple classes inherit from a single parent class.
5. **Hybrid Inheritance**: A combination of more than one type of inheritance.